PDSP 2025, Lecture 04, 19 August 2025

Checking if a number is prime

- Checking if `n` is a prime: assume it is, and flag that is not if we find a factor between `2` and `sqrt(n)`

```
import math
```

```
n = 25
isprime = True
for i in range(2,int(math.sqrt(n))+1):  # int(...) truncates a float to an int
    if n % i == 0:
        isprime = False
```

```
isprime
```

```
False
```

Computing primes upto `n`

- Instead of checking if `n` is a prime, find all primes upto (and including) `n`
- Generate the sequence `2,3,...,n`
- For each element in this sequence, check if it is a prime
- Accumulate all primes found in a list
  - Recall that `l1 + l2` concatenates two lists into a single list
- Two *nested* loops, use different variables `j` and `i` to iterate

```
n = 100
primelist = []
for j in range(2,n+1):
    isprime = True
    for i in range(2,j):
        if j % i == 0:
            isprime = False
    if isprime:
        primelist = primelist + [j]
```

```
primelist
```

```
[2,
 3,
 5,
 7,
 11,
 13,
 17,
 19,
 23,
 29,
 31,
 37,
 41,
 43,
 47,
 53,
 59,
 61,
 67,
 71,
 73,
 79,
 83,
 89,
 97]
```

Appending a value to a list

- Can also use `l.append(v)` to add an element `v` to a list
- Note the distinction between `l + [v]` and `l.append(v)`

- In the first case, we have to make `v` into a singleton list `[v]` to use the operator `+`

```
n = 100
primelist = []
for j in range(2,n+1):
    isprime = True
    for i in range(2,j):
        if j % i == 0:
            isprime = False
    if isprime:
        primelist.append(j)
```

```
primelist
```

```
[2,
 3,
 5,
 7,
 11,
 13,
 17,
 19,
 23,
 29,
 31,
 37,
 41,
 43,
 47,
 53,
 59,
 61,
 67,
 71,
 73,
 79,
 83,
 89,
 97]
```

Functions

- Modularise code into functional units
- Instead of embedding code to check if `j` is a prime, call a function that returns `True` if `j` is a prime and `False` otherwise
- Function definition starts with `def function_name (argument1, argument2, ...):`
- When the function completes, it should report an answer -- return a value through `return(v)`

```
def isprime(n):
    status = True
    for i in range(2,n):
        if n % i == 0:
            status = False
    return(status)
```

```
isprime(17), isprime(25)
```

```
(True, False)
```

Exiting a function in between

- If we find a factor, we can declare the number to not be a prime without testing more factors
- In the original implementation, we needed to exit the loop
- `return()` automatically exits, so we can use this optimisation in the function

```
def isprime2(n): # An equivalent defn, terminates with False at first factor
    status = True
    for i in range(2,n):
        if n % i == 0:
            status = False
            return(status)
    return(status)
```

```
isprime2(47), isprime2(44)
```

```
(True, False)
```

- In fact, we don't even need the variable `status`
- If we find a factor, `return(False)`
- If the search for a factor ends without finding one, `return(True)`

```
def isprime3(n):    # An equivalent defn, without a separate status variable
    for i in range(2,n):
        if n % i == 0:
            return(False)
    return(True)
```

```
isprime3(571), isprime3(573)
```

```
(True, False)
```

Using functions

- We can rewrite our code to search for primes upto `n` to call the function `isprime` for each candidate
  - Recall that in our earlier, explicit, code, we had to rename the outer loop variable as `j` to avoid a clash with the loop through potential factors
  - If we use a function, the `i` inside the function is different from the `i` outside the function

```
n = 100
primelist = []
for i in range(2,n+1):
    if isprime(i):
        primelist.append(i)
```

```
primelist
```

```
[2,
 3,
 5,
 7,
 11,
 13,
 17,
 19,
 23,
 29,
 31,
 37,
 41,
 43,
 47,
 53,
 59,
 61,
 67,
 71,
 73,
 79,
 83,
 89,
 97]
```

- We can convert this search for primes upto `n` into another function

```
def primesupto(n):
    primelist = []
    for i in range(2,n+1):
        if isprime(i):
            primelist.append(i)
    return(primelist)
```

```
primesupto(30)
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

```
primesupto(70)
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67]
```

```
primesupto(1000)
```

```
[2,
 3,
 5,
 7,
 11,
 13,
 17,
 19,
 23,
 29,
 31,
 37,
 41,
 43,
 47,
 53,
 59,
 61,
 67,
 71,
 73,
 79,
 83,
 89,
 97,
 101,
 103,
 107,
 109,
 113,
 127,
 131,
 137,
 139,
 149,
 151,
 157,
 163,
 167,
 173,
 179,
 181,
 191,
 193,
 197,
 199,
 211,
 223,
 227,
 229,
 233,
 239,
 241,
 251,
 257,
 263,
 269,
 271,
```

Functions and modularity

- Functions modularise code
- Each function has an *interface contract* -- if the input x is valid, the output is $f(x)$
- Can change the implementation of the function so long as the interface contract is upheld
  - Any one of our three implmentations of `isprime` can be used
  - For instance, can use a naive implementation as a *prototype* and later replace by a more refined, optimised implementation

First n primes

What if we want a list of the first n primes?

- Generate numbers 2,3,... and check if each one is a prime
- Stop when we have generated n primes

We don't know the upper bound of the list 2,3,...

- Can't use `range()`

Instead, a new kind of loop

- "Manually" generate the sequence
- Stop when we reach the terminating condition

```
while (condition):
  statement 1
  ...
  statement k
```

- If `condition` evaluates to `True` the block of k statements is executed
- After this, the condition is checked again and the same process is repeated
- Compare to `if` where the condition is evaluated once

```
if (condition):
  statement 1
  ...
  statement k
```

```
def nprimes(n):
    primelist = []
    i = 2
    while (len(primelist) < n):
        if (isprime(i)):
            primelist.append(i)
        i = i+1
    return(primelist)
```

```
nprimes(20)
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71]
```

Infinite loops

- Need to ensure that the statements make *progress* towards falsifying the condition
- If the condition remains `True` forever, the loop never terminates
- For instance, suppose there were only finitely many primes, say M. For any $n > M$, the length of `primelist` would saturate at M so the condition `len(primelist) < n` would never become `False`

Looping --- `for` and `while`

- `while` is more general than `for`
- Can implement

  ```
  for x in l:
    ...
  ```

  using `while` by explicitly going through `l` from first to last position

```
pos = 0
while (pos < len(l)):
  ...
  pos = pos + 1
```

- Note that we have to move the position "manually" to ensure that we make progress towards termination
- However, using `for` is preferred if it is clearly an iteration over a fixed sequence
  - The intent is capture much more clearly
  - In the `while` form it is slightly obfuscated

Boolean datatypes

- Usually an outcome of comparisons: `==`, `!=`, `<`, `<=`, `>`, `>=`
- Useful shortcut
  - Any "empty" value is interpreted as `False`
  - So `0`, `[]`, `""` (empty string) are all `False`
  - Any other value is interpreted as `True`

- Avoid comparisons such as `if x == 0` or `if l != []`
  - Write `if not(x)`, `if l` instead

```
l = [1,2,3]
if l:
    x = True
else:
    x = False
```

```
x
```

```
True
```

```
m = 0
if not(m):
    y = True
else:
    y = False
```

```
y
```

```
True
```

- Note that Python does not insist on brackets around the condition in `if` and `while`
  - Can write `if (cond):` or `if cond:`, `while (cond):` or `while cond:`

Variables, values and types

- Variables (names) have no intrinsic types
- Values have types
  - A variable inherits the type of the value it currently holds
- The type of value a variable holds can vary over time
  - But not a good idea to use the same name for different types of values in the same piece of code
  - Reduces readability, maintainability
- The `type()` function returns the type of a variable that is currently assigned a value

```
x = True
```

```
type(x)
```

```
bool
```

```
x = 5
```

```
type(x)
```

```
int
```

- The function `del()` unassigns a value from a name

```
del(x)
```

```
type(x)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[31], line 1
----> 1 type(x)

NameError: name 'x' is not defined
```